

Designed by Remus Teodorescu 2026

LECTURE 3.2

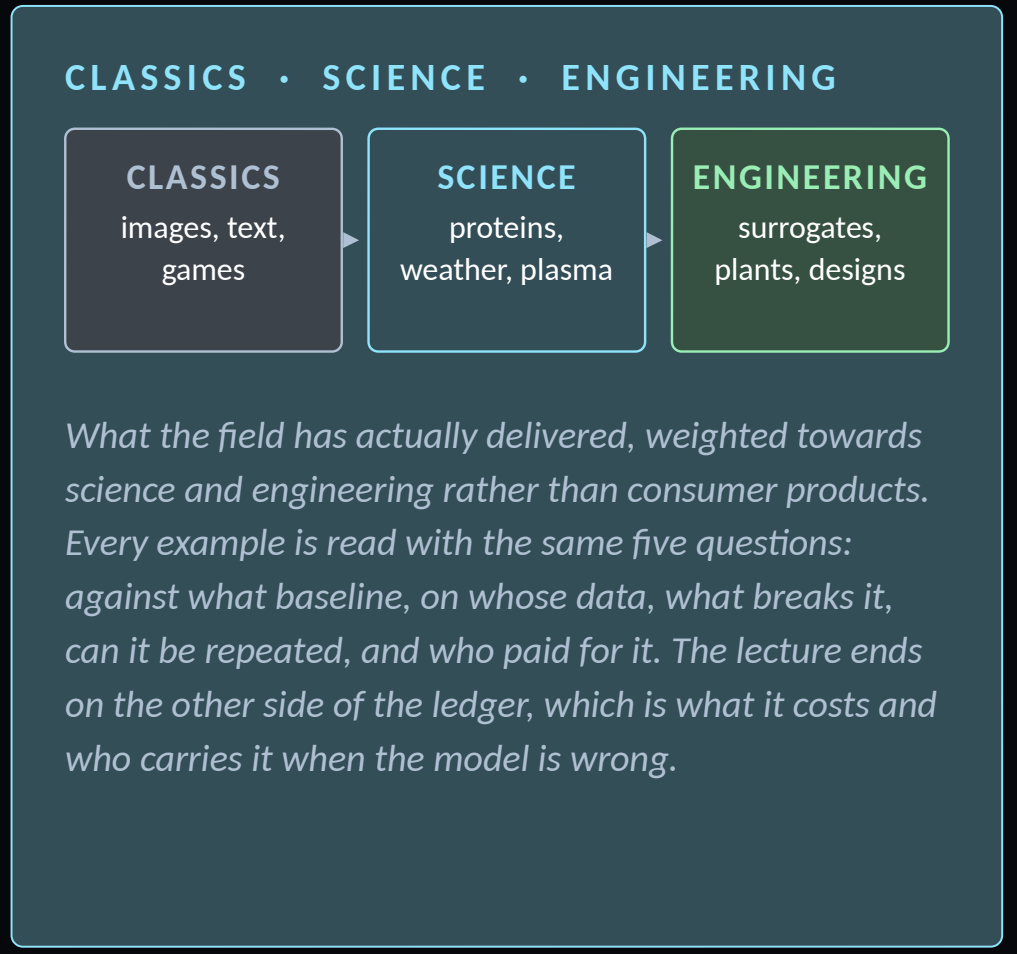
What the Field Has Actually Built

Remus Teodorescu

With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026



What the field has actually delivered, weighted towards science and engineering rather than consumer products. Every example is read with the same five questions: against what baseline, on whose data, what breaks it, can it be repeated, and who paid for it. The lecture ends on the other side of the ledger, which is what it costs and who carries it when the model is wrong.

Learning Objectives

SECTIONS, IN ORDER

1. **The classics** — computer vision, language models, speech and games
2. **Science** — protein structure, weather forecasting, plasma control
3. **Engineering** — surrogates, maintenance, design search, the energy industry, what it costs

Every example is read with the same five questions, and those come first. This lecture is a biased sample: nobody publishes the projects that failed.

READ, AND WHAT EACH ANSWERS

UDL ch. 10, 12	what did the field solve first, and why those?
UDL ch. 13, 19	what has it done that science could not?
UDL ch. 1.4	which of it would you actually build?

OVERVIEW

How to Read a Claim

Reading a claim — the five questions to ask of any reported result before believing it: against what baseline, on whose data, what breaks it, can it be repeated, and who paid for it. They are worth more than any architecture, because they do not change when the architectures do.

FIVE QUESTIONS, ASKED IN THIS ORDER

- **against what** — a real baseline, or one nobody uses
- **on whose data** — the authors' own, or somebody else's
- **what breaks it** — is a failure mode reported at all
- **who paid** — and what would have gone unpublished

APPLY THEM TO THIS LECTURE

baseline	mostly stated — check GraphCast's
failure modes	one slide out of thirty-one
who paid	three of these came from one company

WHY THIS IS THE SKILL

Architectures change every eighteen months and are freely available. These five questions do not change, and they are how every report in this course is marked. Ask them of your own work first — it is cheaper than having someone else ask.

Computer Vision

Computer vision — learning to read images. Three tasks in increasing difficulty: a label for the whole image, a label and a box for each object, and a label for every pixel. What changes between them is the output, not the method, and the last is usually the one an engineer actually wants.

THREE TASKS, ONE METHOD

- **classification** — one label: is there a crack?
- **detection** — a label and a box: where are the cracks?
- **segmentation** — a label for every pixel



THE FOUR QUESTIONS, ANSWERED

input	a grid of pixels, 3 channels
loss	cross-entropy, per image or per pixel
data	hand-labelled — the expensive part
failure	confident label on an unseen defect

TALK · 2 MIN How would you explain the difference between classifying an image and segmenting it, in your own words? Then, in the picture below, which of the three would you need to find those cracks, and who would label them?

Language Models

Language model — a network trained to predict the next piece of text, which is where the field learned to work at scale. The exportable idea is not the language: it is self-supervision, hiding part of the input and predicting it, which needs no labels at all.

THE TASKS, AND THE EXPORTABLE IDEA

- **translation** — the task that motivated the transformer
- **then everything else** — summaries, answers, code
- **self-supervision** — hide part of the input, predict it

THE FOUR QUESTIONS

input	a sequence of tokens
loss	predict the next token
data	scraped text — legally contested
failure	fluent, confident, false

SELF-SUPERVISION, THE EXPORTABLE IDEA

Hide part of the input and predict it. No labels required. For an engineer sitting on a year of unlabelled sensor data and no fault annotations, that is not a curiosity — it is the most promising thing on this slide.

Speech and Games

Learning to act — learning a policy by trial and error against a score, which is what games gave the field. The interesting part for an engineer is what happened next: the same method moved from boards to physical systems, to cooling a data centre and to holding a plasma off a tokamak wall.

FROM A BOARD TO A PLANT

- **speech** — solved around 2012, and now invisible
- **AlphaGo, 2016** — more positions than atoms in the universe
- **then physical systems** — cooling, chips, plasma

GAME TO PLANT, IN FOUR STEPS

2016	AlphaGo — a board, and a win/loss
2017	self-play, no human games needed
2020	data-centre cooling in production
2022	tokamak plasma control, on hardware
your turn	L11, on a car that can crash

THE STEP THAT IS ACTUALLY HARD

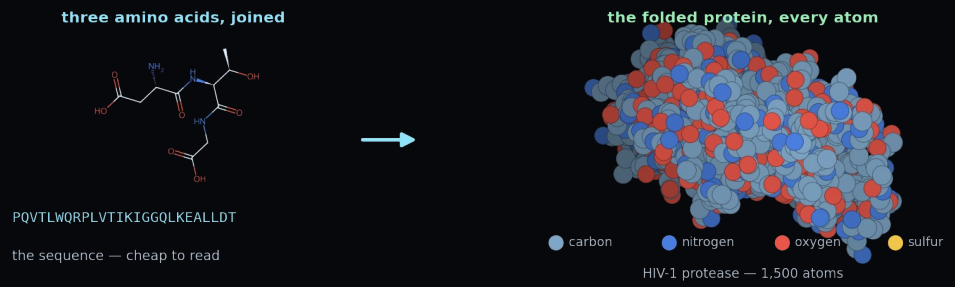
Not the algorithm — the simulator. A policy learned in simulation and deployed on hardware meets every modelling error at once. That gap has a name, the sim-to-real gap, and L11 is where you will feel it.

Protein Structure

Protein structure prediction — working out the three-dimensional shape a chain of amino acids folds into, because the shape is what decides the function. It was open for fifty years, not for want of physics but because the space of possible shapes is astronomically large, and it was effectively solved in 2021.

WHY IT WAS HARD, AND WHAT SOLVED IT

- **the forces were known** — the search space was not tractable
- **the data** — fifty years of measured structures, curated
- **the result** — accuracy comparable with experiment



THE FOUR QUESTIONS

input	amino acid sequence, plus relatives
loss	distance and angle error vs. structure
data	decades of public crystallography
failure	confident on disordered regions

THE LESSON FOR AN ENGINEER

Fifty years of publicly deposited experimental structures is what made this possible. The model is remarkable; the dataset was the precondition. When you read a result like this, ask who spent fifty years collecting the data.

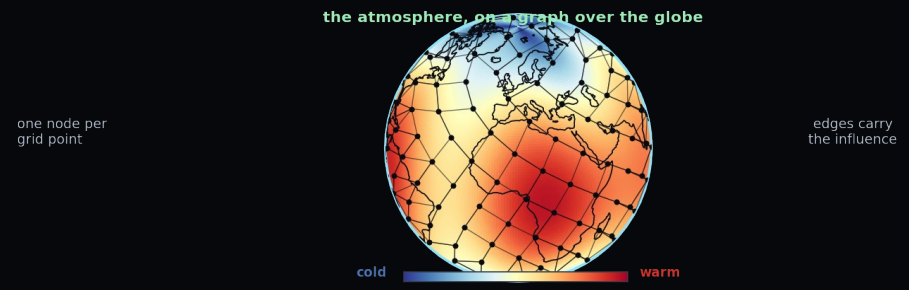
Jumper et al. (2021), Nature 596

Weather Forecasting

Learned weather forecasting — predicting the atmosphere with a network trained on decades of reanalysis data instead of integrating the equations of motion. GraphCast, published in 2023, beat the leading physical model on most measures at a fraction of the cost, and it still starts from that model's analysis.

THE ONE THIS ROOM SHOULD READ CLOSELY

- **the baseline** — the best physical model there is
- **the result** — better on most variables, in one minute
- **the dependency** — it starts from a physical analysis



THE FOUR QUESTIONS

input	gridded atmospheric state, 2 times
loss	weighted error at future lead times
data	ERA5 reanalysis — decades, public
failure	unprecedented events, poorly

WHAT IT DOES NOT DO

It does not replace the physics. It was trained on reanalysis, which is itself the output of a physical model fused with observations. Remove the physics-based pipeline and there is no training data. Say that when someone tells you weather forecasting has been solved by AI.

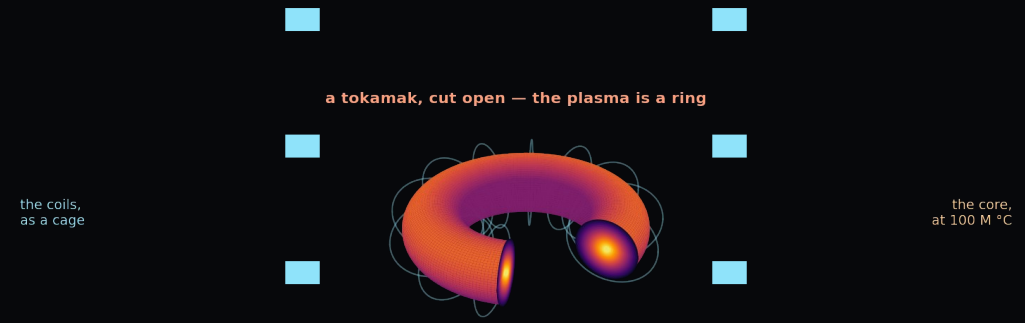
Lam et al. (2023), Science 382

Plasma Control

Plasma control — holding a plasma of a hundred million degrees away from the wall of a tokamak with magnetic fields. Conventionally each configuration needs its own controller, tuned over weeks. One reinforcement-learning policy, trained in simulation, controlled several of them on real hardware in 2022.

A CONTROL PROBLEM, SOLVED THE HARD WAY

- **conventionally** — one controller per configuration, tuned
- **instead** — one policy, trained in a simulator
- **then run on hardware** — which is the difficult part



held clear of the wall by the field alone — and the balance drifts in milliseconds

THE FOUR QUESTIONS

input	magnetic measurements, real time
loss	reward on shape and position error
data	generated in a physics simulator
failure	the plasma touches the wall

WHY THIS RESULT MATTERS HERE

Almost no measured data. A trusted physical simulator. A control problem where failure is expensive and exploration is forbidden. That is a very common engineering situation, and it is precisely the situation L11 puts you in — with a cheaper car.

Degrave et al. (2022), Nature 602

Surrogate Models

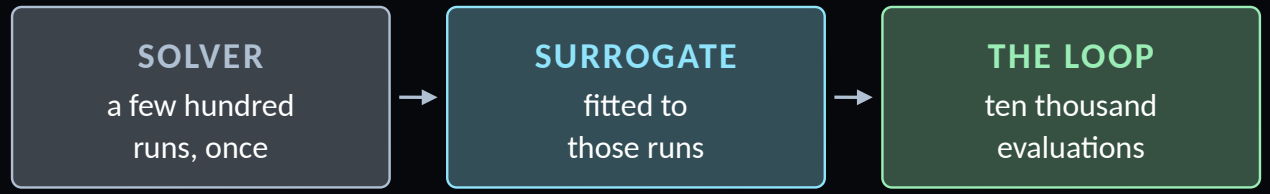
Surrogate model — a network trained on the outputs of a solver you already trust, and then used in its place. You run the solver a few hundred times, fit the network to those runs, and evaluate it in microseconds instead of hours. It is the most common engineering use of deep learning by a wide margin.

THE TRADE, AND ITS LIMIT

- **the situation** — a trusted solver, and a loop of 10 000 runs
- **the trade** — a few hundred runs once, microseconds after
- **the limit** — it is only trustworthy inside the range you sampled

$$f_\theta(x) \approx \text{Solver}(x) \qquad t_f \ll t_{\text{Solver}}$$

f_θ the trained network, x the design variables, t the time of one evaluation



Pay once, then evaluate ten thousand designs. That trade is the whole idea.

THE FOUR QUESTIONS

input	design variables, or a whole field
loss	error against the solver's output
data	your own solver — you make it
failure	silently wrong off the sample set

TALK · 2 MIN In your own work, what would you put a surrogate in front of, and what would it have to be right about? Then, in the figure on the left, how many solver runs would you spend before it pays for itself?

THE SURROGATE, IN TWO LINES

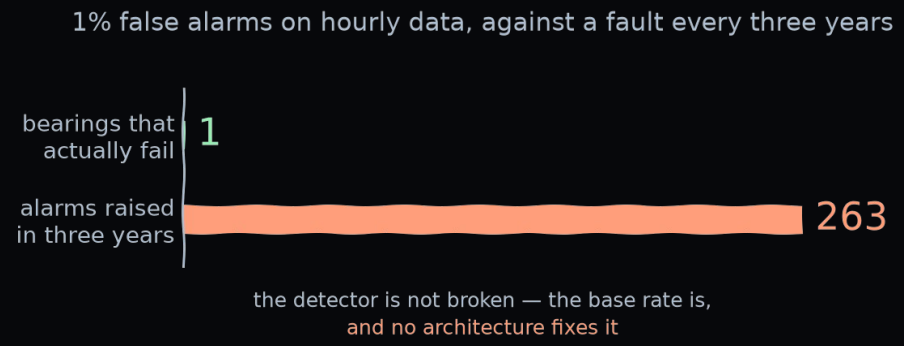
```
net = train(MLP(), X, Y)   # fit to 300 solver runs
y = net(x_new)             # microseconds, not hours
```

Predictive Maintenance

Predictive maintenance — learning what normal looks like from the sensor data a plant already logs, then flagging departures from it. The data is plentiful and the labels are almost absent, because the faults are rare. The difficulty is not the architecture. It is the base rate.

THE PROBLEM IS NOT THE MODEL

- **the structure** — much data, almost no labelled faults
- **the difficulty** — base rates, not the architecture
- **no network fixes this** — it is a statistics problem



WHY THESE PROJECTS STALL

labels	the faults were never annotated
base rate	rare events, many false alarms
drift	the plant changes; the model does not
trust	one bad alarm and it is ignored
ownership	nobody maintains the model

Every item on that list is organisational rather than mathematical. If you take on one of these projects, budget your effort accordingly — and notice that none of the five is taught in a deep learning course, including this one.

Design Search

Design search — searching a space of shapes or layouts for the one that best meets an objective. The search itself is classical and physics-based. What a network contributes is speed: a surrogate in the inner loop, a compact description of the family of shapes, or a good starting point.

WHAT IS CLASSICAL HERE

- **topology optimisation** — remove material where stress is low
- **adjoint shape optimisation** — the aerodynamicist's tool
- neither needs a network, nor is improved by adding one

A TERMINOLOGY TRAP

"Generative design" in CAD is topology optimisation, which is older than deep learning and not generative AI. The words collided, the methods did not.

WHERE LEARNING ACTUALLY HELPS

Speeding up the inner loop with a surrogate. This is the real contribution, and it is the previous slide.

Learning a compact description of a family of shapes, so that the search happens in ten dimensions instead of ten thousand.

Predicting a good starting point from previous designs, so the classical optimiser converges in a fraction of the iterations.

In all three the classical method is doing the engineering. The network removes the cost.

AI in the Energy Industry

AI in the energy industry — what is actually running, as opposed to what is announced. The pattern is the same in every case: the data already existed, the labels did not, and the gain is reported in money or in availability rather than in accuracy.

WHAT THE DEPLOYED CASES HAVE IN COMMON

- **the data was already logged** — nobody collected it for this
- **almost none of it is labelled** — so the learning is self-supervised
- the number reported is money or availability, not accuracy

READ THE LAST ROW TWICE

Breathe Charge is not machine learning. It is an electrochemical estimator, and it beat the static tables. Not every gain here is a network.

RUNNING NOW, WITH A NUMBER

Energinet	a forecast sets the daily reserve volume, about 50 M DKK a year
Eur. Energy	Kassø e-methanol, 52 MW, running largely autonomously
Ørsted	5.5 GW of wind and solar under one model, on unlabelled SCADA
Vestas	reinforcement learning steers wakes across a whole farm
Grundfos	vibration and temperature on pumps, then sold as a service
Breathe Charge	no learning at all: an electrochemical estimator, in Volvo cars

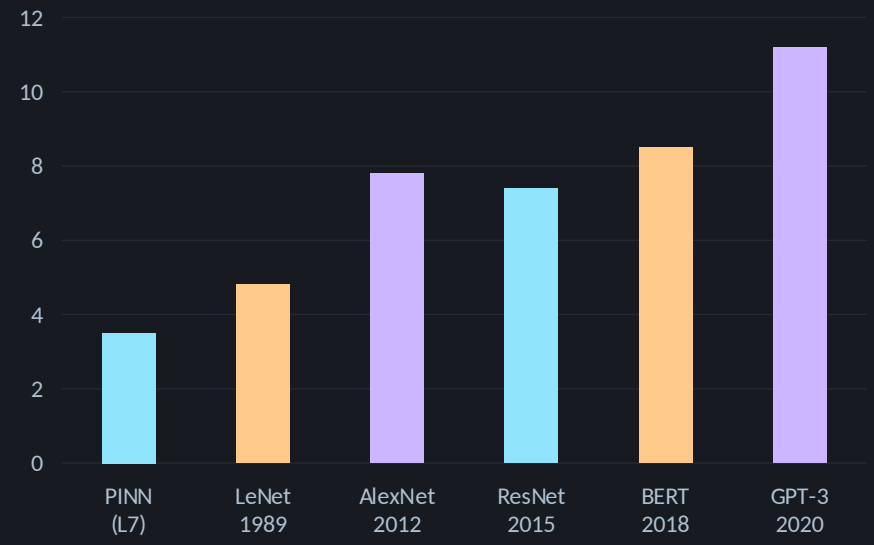
What It Costs

What it costs — the other side of the ledger: the energy and money spent training large models, the data taken to do it, and the question of who carries the consequence when a model is wrong. Almost none of the compute cost applies to the models in this course, and all of the responsibility does.

THE NUMBER NOBODY PUTS ON THE SLIDE

- **training compute** — orders of magnitude in a decade
- **not your problem** — every model here trains in minutes
- **your problem** — who carries it when the model is wrong

PARAMETERS, LOG SCALE, ILLUSTRATIVE



Order-of-magnitude figures for teaching, not a citable table. The left-hand bar is about three thousand parameters — the size of the PINN you build in L7.

Advanced Topics

APPLICATION AREAS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
recommender systems	ranking what a person sees next	UDL ch. 1 notes	the largest deployed use there is
autonomous driving	perception and planning together	UDL ch. 10, 19	the most heavily funded area
medical imaging	diagnosis from scans, and its approval	UDL ch. 10.5	where regulation is hardest
molecule design	generating candidates, not reading them	UDL ch. 14 – 18	pharmaceutical research
foundation models for physics	one model across many equations	UDL ch. 12.6 – 12.7	an open research question

WHY THEY ARE LEFT OUT

Each is an application area rather than a method, and the methods underneath them are the ones this course teaches. Adding these five would add domains, not understanding, and the examples that remain are the ones an energy engineer meets.

EXERCISE

Exercise 3

WHAT THE NOTEBOOK DOES

The same notebook as the last lecture, read with this lecture's questions. A mass on a spring, measured for five seconds by a sensor that is noisy and was never zeroed, modelled five ways.

For each of the five, answer the five questions on the third slide. What is the input. What is the loss. Where did the data come from. What happens when it is wrong. And what would you have to show somebody before they believed it.

Two of the five models fit the measurements better than the model that is right. That is the whole lecture in one notebook, and it is why the questions come before the architectures.

THE QUESTION IT ANSWERS

Ex_03	which claim would you believe?
models 1 – 3	what does a good fit hide?
models 4 – 5	what does the physics buy?

REPORT QUESTION, TO CONCLUDE

Take one result from today and one from your own field, and ask the five questions of both. Which was hardest to answer, and what does that tell you?

SUMMARY

Key Takeaways

WHAT IT HAS BUILT

what it is	results in science, read with the five questions
function	reads images, text, and physical fields
train, tune	the data existed first, the method came second
vs physics	better on most variables, in one minute
used for	protein structure, weather, plasma control

WHAT AN ENGINEER BUILDS

what it is	a surrogate, a monitor, or a search
function	replaces a solver, a rule, or a sweep
train, tune	a few hundred solver runs, or logs you have
vs the solver	microseconds not hours, inside its range
used for	design loops, plant monitoring, shape search

Questions

1 – 2 · WHAT IT HAS BUILT

1 **What are the five questions to ask of any reported result?**

Against what baseline, on whose data, what breaks it, can it be repeated, and who paid for it. They test whether the comparison is fair, whether the result holds on data the authors did not choose, and whether anyone else could confirm it. *UDL ch. 1.4*

2 **What do AlphaFold, GraphCast and the tokamak controller have in common?**

In each, a large body of data or a simulator existed first and the network learned from it: measured protein structures, decades of reanalysed weather, a plasma simulator. Each beat a strong baseline on a stated measure, and each still depends on the physics that made its data.
Jumper et al. (2021), Nature 596; Lam et al. (2023), Science 382

3 – 4 · WHAT AN ENGINEER BUILDS

3 **What is a surrogate model, and where does it stop being trustworthy?**

A network trained on a few hundred runs of a solver to give the same answer in microseconds instead of hours. It pays when the solver must be called many times, as in a design loop. It is trustworthy only inside the range of inputs it was trained on. *UDL ch. 1.4*

4 **What can a network do for an engineer besides replacing a solver, and what stays the engineer's work?**

Monitor a plant from the logs it already has, where the difficulty is that faults are rare, and search a design space faster than a sweep. The engineer still chooses the objective, the constraints and the check that says the answer is right. *UDL ch. 1.4*

NEXT

L4.1 — From Perceptron to Neural Network

Two lectures of framing are done. Next time we build the thing itself, starting from a straight line and ending with a proof that a shallow network can approximate anything — and an explanation of why that proof matters less than it sounds.

BEFORE THE NEXT SESSION

read	UDL ch. 2 and 3 — about an hour
run	Ex_03, the one notebook
bring	your extrapolation plot from Ex_03
think about	why the network extrapolated worst

Bring the plot. L4.1 opens by explaining what you will have seen, and the explanation is much more convincing if you have already been surprised by it once.